To use tags, you need to define them by writing additional configuration code, presumably in the same file where you put the basic configuration. The tag configuration will look like this:

```
VCR.cucumber_tags do |t|
  t.tag '@vcr', use_scenario_name: true
  t.tags '@twitter', '@facebook'
end
```

Any options you would pass to use_cassette can be passed as key/value pairs after the tag name. If you want to define multiple tags at once, you can use the tags method.

The tags can then be used like normal Cucumber tags:

```
@vcr
Scenario: Get the user's Twitter avatar
  Given a logged-in user
  When that user has completed a task
  And I view the project page
  Then I should see the user's Twitter avatar
```

When a VCR-related tag is used for a Cucumber scenario, VCR is activated for the duration of the scenario. The resulting cassette file is named after the tag unless the use_scenario_name: option is true, in which case VCR generates a name based on the feature and scenario names, similar to how it generates a name when using RSpec metadata.

Client Unit Tests

VCR is set up; now let's make the Twitter integration work. As it stands, the test fails because the user data is not in the view at all. The test suggests that the user email and the Twitter avatar should be in the view, so let's add them to the view file.

You have a design decision to make about how the application should interact with Twitter. There are many options, ranging from calling the gem and service directly from the view to placing the interaction within the User class.

My design here tends toward more objects and structure on the grounds that we're using this avatar to stand in for a more complex third-party integration. The set of classes might feel like overkill, but I want to demonstrate what this is like with all the moving parts.

With that throat-clearing out of the way, let's write some tests. It seems like getting the Twitter avatar-image URL is a User responsibility.